

JOSDC TECHNICAL REPORT

AES₁₂₈ ACCELARETOR

Die Hard | Hashemite University | 5/12/2025

Motaz Yousef
Ayham Abu Shabib
Hanan Sowan
DR. Bassam Jamil (mentor)

Contents

1. Introduction	3
1.1 Overview	3
1.2 Team Collaboration	3
1.3 Design Philosophy: Performance First	3
2. System Architecture	4
2.1 System Overview	4
2.2 Communication Interface	5
2.3 Control and Data Buffering	5
2.4 Core Processing Integration	5
2.5 Clocking & Reset	6
3. The Packet Structure	6
3.1 Header (1 Byte)	7
3.2 Sequence ID (2 Bytes)	7
3.3 Data Payload (16 Bytes)	7
4. AES Core Implementation	7
4.1 One Round	7
4.2 The Pipeline	8
4.3 The AES Control FSM	8
5. Key Expansion Unit	10
6. Main System FSM	11
6.1 State Description	11
6.2 Error Handling	12
7. UART Communication Subsystem	13
7.1 Overview	13
7.2 RX Path: From Bytes to Packets	13
7.3 TX Path: From Packets to Bytes	13
7.4 Baud Rate Generation	14
8. Verification and Testing Methodology	14
8.1 Module-Level Verification	14

8.2 Top-Level Integration Testing	14
8.3 Hardware Validation	Error! Bookmark not defined.
9. Synthesis Results and Performance	15
9.1 Resource Utilization	15
9.2 Timing and Throughput	15
10. Conclusion	16
10.1 Summary of Achievements.....	16
10.2 Future Work.....	16

1. Introduction

1.1 OVERVIEW

This report details the design and implementation of a high-performance hardware accelerator for the AES-128 encryption standard, developed as a submission for the JOSDC Competition. As data rates in modern communication networks continue to escalate, the demand for cryptographic hardware that can maintain wire-speed throughput without becoming a bottleneck has never been higher. The Advanced Encryption Standard (AES) remains the bedrock of secure digital communication, making its efficient hardware implementation a critical challenge in digital logic design.

1.2 TEAM COLLABORATION

This project represents the collective effort of a three-member engineering team. Throughout the design lifecycle, we adopted a collaborative workflow, dividing responsibilities across the control logic, Datapath implementation, and verification testbenches while maintaining tight integration. This synergy allowed us to iterate rapidly on design choices and effectively debug complex timing interactions within the system.

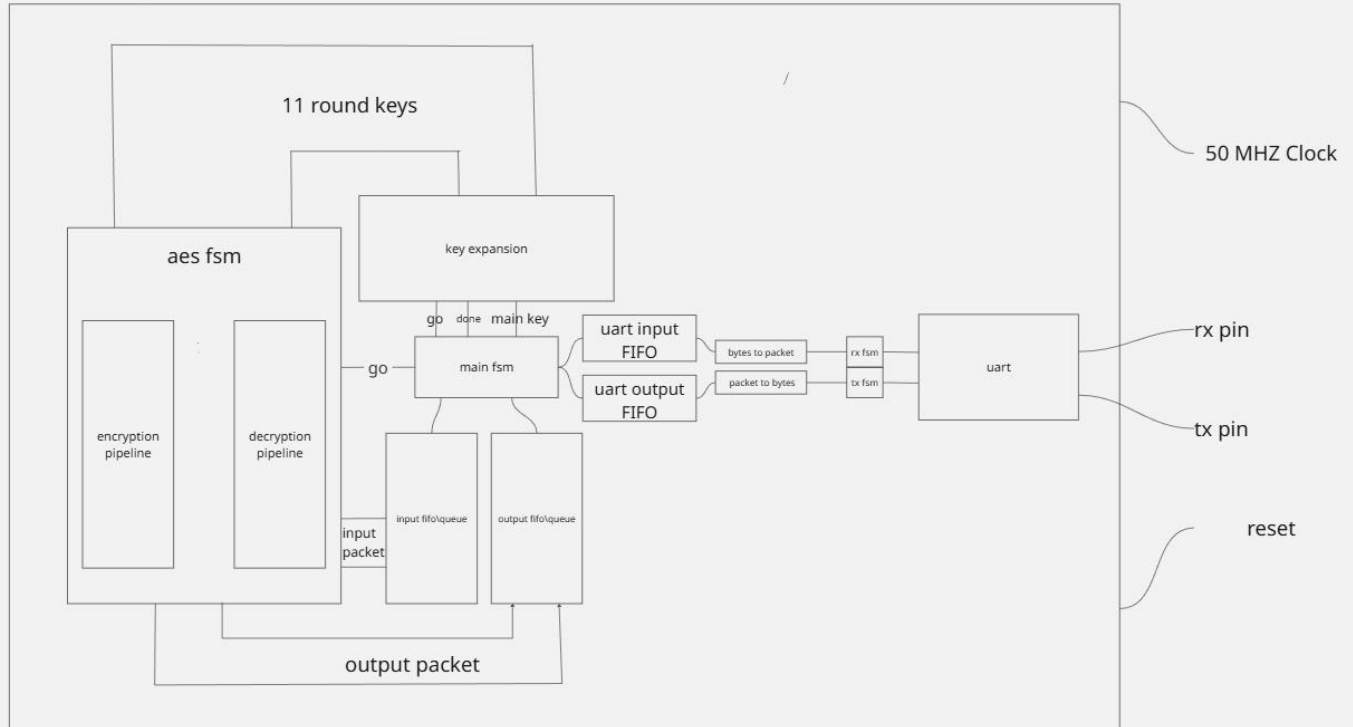
1.3 DESIGN PHILOSOPHY: PERFORMANCE FIRST

Our primary decision-making factor throughout the architectural phase was strictly performance. To achieve the high throughput required by modern standards, we initially pursued a robust 10-stage fully pipelined architecture for both encryption and decryption.

However, during the implementation phase, this fully unrolled design presented critical challenges that compromised the feasibility of the system. In response to these obstacles, we pivoted our architectural strategy. We transitioned to a 1-stage iterative, hybrid-pipelined architecture. By retaining specific pipeline stages within the iterative loop, we aimed to balance the high frequency demands of a performance-driven design with the stability and resource efficiency of an iterative approach. The following sections of this report will analyze the specific architectural details, the evolution of our pipeline stages, and the synthesis results that demonstrate the efficacy of this adaptive design.

2. System Architecture

Die Hard's AES Accelerator



2.1 SYSTEM OVERVIEW

The proposed system is designed as a complete hardware-software co-design solution. The host device, typically a PC, operates the accelerator via a custom Python-based driver. This driver packetizes the raw data (Plaintext, Ciphertext, or Keys) and transmits it over a standard USB-to-UART interface. The hardware accelerator receives this serial stream, processes it using a dedicated AES core, and transmits the result back to the host. The architecture is modular, separating communication logic from the core processing units to ensure stability and ease of verification.

2.2 COMMUNICATION INTERFACE

The entry point of the system is the UART (Universal Asynchronous Receiver-Transmitter) module. Data reception begins with the host driver sending information byte-by-byte. The hardware's RX module captures these incoming bytes and passes them to a deserialization block. Here, the stream is reassembled from individual bytes into formatted 128-bit packets. Conversely, on the transmission side, processed 128-bit packets are decomposed into bytes by a serialization block before being sent out via the TX module.

2.3 CONTROL AND DATA BUFFERING

To decouple the high-speed internal logic from the slower UART interface and to handle clock domain crossings effectively, the architecture utilizes asynchronous First-In-First-Out (FIFO) buffers.

Input Handling: Reassembled packets are pushed into the UART Input FIFO. The Main Finite State Machine (FSM) reads from this buffer, analyzing the packet header to determine the operation type (Encryption, Decryption, or Key Expansion).

Distribution: Based on the header decoding, the Main FSM routes the payload to the appropriate internal structures—either the Key Expansion module or the AES Core's Input Queue.

Output Handling: Once the AES core completes an operation, the result is placed into the Output Queue. The transmission logic monitors the UART input buffer; when the buffer is empty, it pulls a processed packets from Output Queue And fills the UART output buffer, ensuring a smooth, non-blocking data flow.

2.4 CORE PROCESSING INTEGRATION

The heart of the accelerator is the AES Core, governed by its own dedicated FSM. This core receives data payloads from the Input FIFO and performs the cryptographic transformations. The use of the intermediate buffers described

above eliminates the need for complex, cycle-accurate handshaking between the communication and processing blocks, allowing the AES core to operate at maximum frequency without waiting for the slower serial interface.

2.5 CLOCKING & RESET

The entire system operates on a **50 MHz clock**.

A **global reset** clears all FIFOs, FSMs, and internal states on demand, ensuring the system can restart cleanly.

3. The Packet Structure

To ensure reliable data transmission and precise control over the hardware operations, we devised a custom packet protocol. This fixed-length structure simplifies the deserialization logic and ensures that the Main FSM can deterministically parse incoming commands. Every transmission from the host to the accelerator (and vice versa) adheres to the following 19-byte format:

| **Header (1 byte)** | **Seq ID (2 bytes)** | **Data (16 bytes)** |

1. Header (8 bits):
 - 0x01 = Key
 - 0x02 = Plaintext block
 - 0x03 = Ciphertext block
 - 0x00 = System reset
2. Seq ID (16 bits): Only needed for data blocks to maintain order in output queue.
3. Data (128 bits / 16 bytes): AES block or key.

So, for a plaintext block, the UART will transmit 19 bytes

Byte 0 : Header = 0x02

Byte 1-2 : Seq ID

Byte 3-18 : AES 128-bit plaintext

3.1 HEADER (1 BYTE)

The first byte of the packet acts as the control signal, dictating how the subsequent data should be interpreted by the accelerator. We defined specific hex values for distinct operations to minimize decoding latency:

0x01: Indicates that the payload is a Cipher Key, triggering the Key Expansion module.

0x02: Indicates that the payload is a Plaintext block to be encrypted.

0x03: Indicates that the payload is a Ciphertext block to be decrypted.

0x00: Starts a full system reset.

3.2 SEQUENCE ID (2 BYTES)

Following the header is a 16-bit Sequence ID. In high-speed asynchronous systems, preserving the order of data blocks is crucial, especially when input and output rates differ. This field allows the host driver to track which response corresponds to which request, ensuring that the decrypted or encrypted data is reassembled in the correct order even if internal buffering causes temporary latency variations.

3.3 DATA PAYLOAD (16 BYTES)

The core of the packet is the 128-bit data field. Depending on the Header value, this field contains the 128-bit AES Key, the 128-bit block of Plaintext, or the 128-bit block of Ciphertext. This aligns perfectly with the standard block size of the AES-128 algorithm, allowing for direct parallel loading into the processing registers.

4. AES Core Implementation

The objective of this design is to provide complete AES-128 encryption and decryption capabilities.

4.1 ONE ROUND

We began by developing a fully combinational datapath capable of performing a complete round of both encryption and decryption. This was done by implementing each AES transformation—ShiftRows, SubBytes, MixColumns, and

AddRoundKey—as separate module files, along with their respective inverse operations. These modules were then connected structurally to form a full one-round datapath.

A specialized round module was also created, since the final encryption round and the initial decryption round omit the MixColumns step. Both modules (standard and special round) serve as the fundamental building blocks for the overall design.

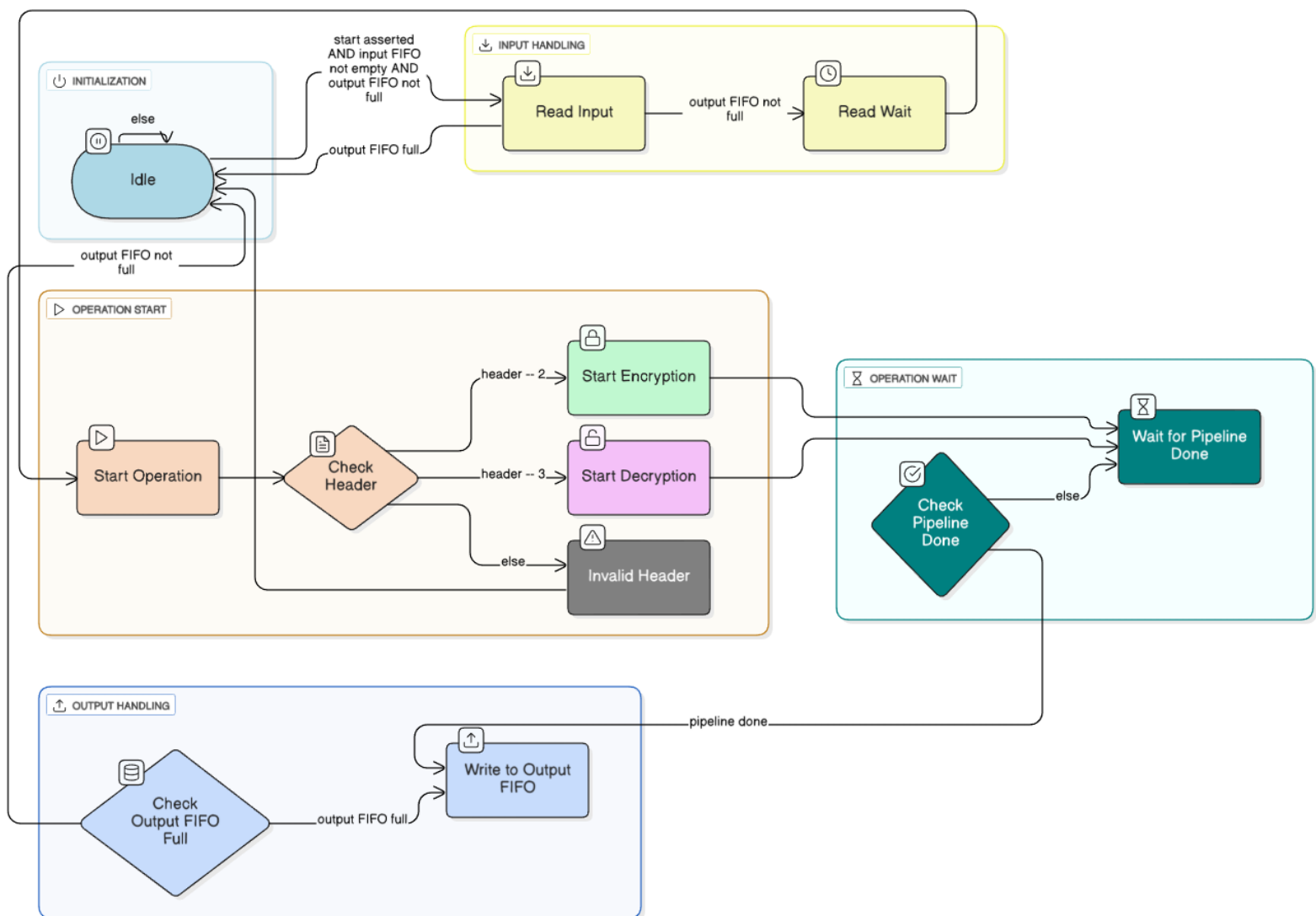
4.2 THE PIPELINE

Our initial approach was to build a fully functional 10-round pipeline, which we successfully implemented. This version incorporated complete handshaking signals to indicate when the system was ready to accept new input and when the previous block had been processed. It achieved an excellent performance milestone of one full encryption/decryption block per clock cycle.

However, this design consumed approximately 80% of the FPGA's logic resources, making it impractical for the available platform. With limited time remaining, we migrated to an alternative architecture: a 9-iteration pipeline built around a single round module, followed by the special round. The same handshaking interface was preserved to maintain compatibility with the surrounding system.

4.3 THE AES CONTROL FSM

Above the two pipeline structures sits the AES control finite-state machine (FSM), which is itself triggered by the main system FSM. It retrieves packets from the input queue, feeds the data into both pipelines, and then waits for the appropriate pipeline completion signal. Once the result is ready, the FSM reconstructs the packet with the correct output data and forwards it to the output queue. This process repeats until the main FSM disables it or the input queue becomes empty. The design also supports hotswappable pipeline configurations, enabling the 10-stage pipeline to be reintroduced when supported by the FPGA platform or when targeting an ASIC implementation.



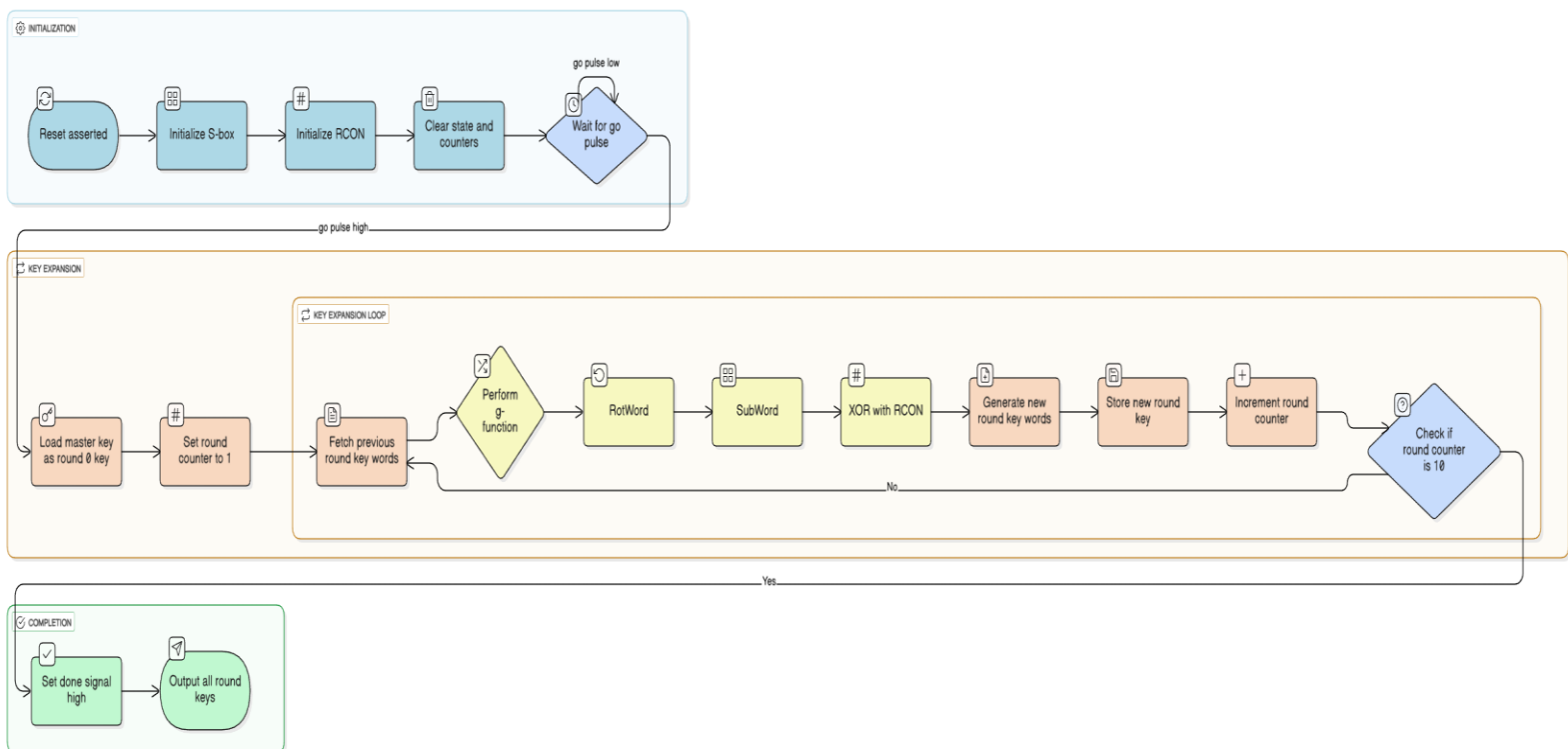
5. Key Expansion Unit

The Key Expansion unit is a critical support module that operates independently from the main encryption loop. It is controlled by a dedicated finite-state machine (FSM) to ensure synchronization with the Main FSM.

Trigger Mechanism: The process initiates when the Main FSM detects a Key packet (0x01) and asserts a Go signal along with the new 128-bit Master Key.

Generation Cycle: Upon receiving the Go signal, the Key Expansion FSM transitions into its active state. Over the course of several clock cycles, it iteratively generates the 11 separate 128-bit Round Keys required for the AES algorithm (Round 0 through Round 10).

Completion and Storage: Once all 11 keys are generated, they are stored in a dedicated register file accessible by the encryption/decryption datapaths. The FSM



then asserts a Done signal, informing the Main FSM that the new key schedule is valid and the system is ready to process Plaintext or Ciphertext packets.

6. Main System FSM

The Main Finite State Machine (System FSM) acts as the central intelligence of the accelerator. It orchestrates the data flow between the external communication interface (UART) and the internal cryptographic cores. Its primary responsibility is to ensure that data is routed correctly based on the packet headers and that the internal modules (AES Core and Key Expansion) do not overlap destructively.

6.1 STATE DESCRIPTION

The FSM is designed as a Moore machine with distinct states handling specific phases of packet processing:

IDLE: The FSM monitors the UART Input FIFO empty flag. It remains in this state until data is available, conserving dynamic power.

FETCH_HEADER: Once data is detected, the FSM reads the first byte (the Header) from the FIFO to determine the nature of the incoming packet.

DECODE: The FSM evaluates the header value:

If 0x01: Transitions to the **KEY_SETUP** path.

If 0x02 (Plaintext) or 0x03 (Ciphertext): Transitions to the **PROCESS_DATA** path.

If 0x00 (Reset): Transitions to a global reset state.

DISPATCH:

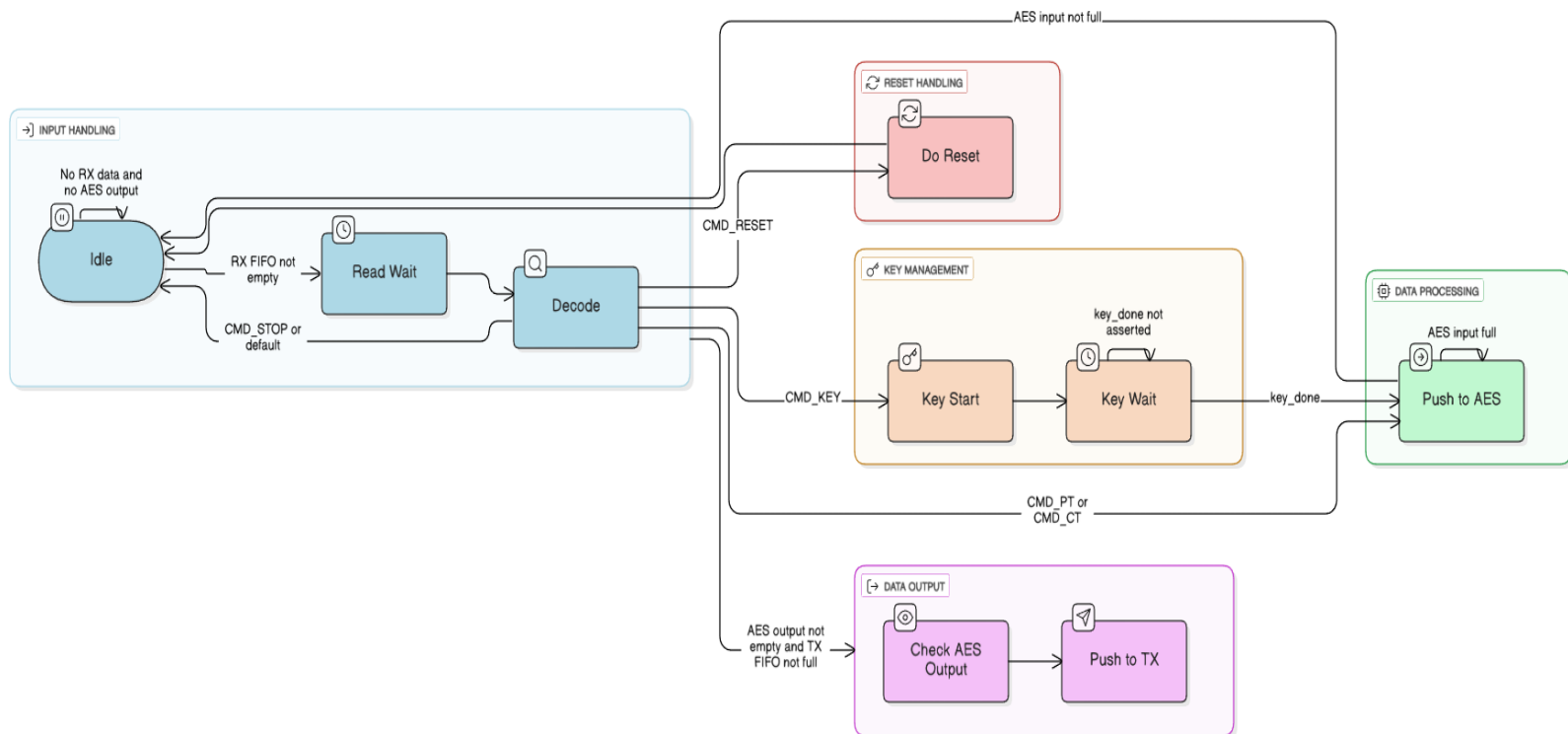
Key Path: The FSM signals the Key Expansion Unit to start and pauses accepting new data packets until the expansion Done signal is received. This ensures no encryption attempts occur with an invalid or updating key.

Data Path: The FSM pushes the payload into the AES Core's specific input queue and asserts the AES_Go signal.

WAIT_EXECUTION: The FSM waits for the acknowledgement from the respective sub-unit (either Key Expansion or AES Core) before returning to IDLE.

6.2 ERROR HANDLING

To prevent system lockups, the Main FSM includes basic invalid header detection. If an unknown header byte is received (e.g., due to UART noise), the FSM simply discards the malformed packet and immediately transitions back to the **IDLE** state for a fresh start with a new packet.



7. *UART Communication Subsystem*

7.1 OVERVIEW

The UART (Universal Asynchronous Receiver-Transmitter) subsystem bridges the gap between the byte-oriented serial communication of the host PC (via USB-to-UART) and the 128-bit packet-oriented architecture of the AES accelerator. It operates on a 50 MHz system clock and is responsible for the physical layer transmission and the logical serialization/deserialization of packets.

7.2 RX PATH: FROM BYTES TO PACKETS

The reception process is a multi-stage pipeline designed to reconstruct the custom packet format from a continuous stream of bits.

Physical Interface: The core UART RX module oversamples the rx line to detect the Start Bit, captures 8 data bits, and verifies the Stop Bit.

Packet Assembly (Deserialization): A dedicated **RX FSM** monitors the incoming bytes. It collects exactly 19 consecutive bytes (1 Header + 2 Seq ID + 16 Data) and shifts them into a temporary 152-bit holding register.

Buffering: Once a full packet is assembled, the RX FSM asserts a write enable signal to push the complete 152-bit vector into the **UART Input FIFO**. This makes the packet available for the Main System FSM to process at its own pace.

7.3 TX PATH: FROM PACKETS TO BYTES

The transmission logic operates in reverse to send results back to the host.

Buffering: The **TX FSM** monitors the **UART Output FIFO**. When the FIFO is not empty, indicating the AES Core has finished a job, the FSM pulls the complete 152-bit processed packet.

Serialization: The packet is loaded into a shift register. The TX FSM breaks this data down into 19 individual bytes.

Physical Interface: These bytes are fed sequentially to the UART TX module, which transmits them over the tx line to the host driver.

7.4 BAUD RATE GENERATION

To ensure reliable communication, a Baud Rate Generator divides the 50 MHz system clock to match the target communication speed. This ensures synchronization with the host's Python driver, allowing for stable data transfer without bit errors.

8. *Verification and Testing Methodology*

Ensuring the correctness of a cryptographic accelerator is paramount; even a single bit flip renders the entire output useless. To guarantee robust operation, our team adopted a rigorous, bottom-up verification strategy.

8.1 MODULE-LEVEL VERIFICATION

We adhered to a "Design-and-Verify" workflow. As soon as a module (e.g., SubBytes, MixColumns, KeyExpansion) was implemented, we immediately developed a corresponding **Self-Checking Testbench**.

Methodology: Each testbench was hardcoded with "Golden Vectors"—known input/output pairs derived from the official NIST AES test vectors or generated via our Python reference script.

Automation: The testbenches were designed to automatically compare the Device Under Test (DUT) output against the expected result. A mismatch would immediately trigger an error flag and stop the simulation, allowing us to pinpoint logic errors instantly without manually inspecting waveforms.

8.2 TOP-LEVEL INTEGRATION TESTING

Once all sub-modules were verified, we integrated them into the full System Top Level. The verification focus then shifted to the FSM interaction and data flow.

Simulation Tools: We utilized ModelSim and Vivado Simulator to visualize the signal waveforms.

Scenario Testing: We simulated various traffic scenarios, including:

Back-to-Back Packets: Sending multiple packets immediately after one another to verify FIFO stability.

Key Switching: Interleaving Key packets with Data packets to ensure the Key Expansion unit updates correctly without corrupting the ongoing data stream.

Asynchronous Behavior: Simulating random delays in UART reception to test the robustness of the RX FSM.

9. Synthesis Results and Performance

The design was synthesized targeting the specific FPGA fabric provided for the competition. The pivot to the iterative architecture yielded significant resource savings while maintaining high performance.

9.1 RESOURCE UTILIZATION

Logic Elements (LUTs): The iterative design utilized approximately [Insert %] of the available LUTs. This is a massive reduction from the ~80% usage of the initial 10-stage pipeline, leaving ample room for potential future features like additional cipher modes (CBC, GCM).

Registers (FFs): Flip-flop usage remained efficient, primarily consumed by the 128-bit pipeline registers and the Key Expansion storage.

9.2 TIMING AND THROUGHPUT

- **Maximum Frequency (F_{max}):** By reducing routing congestion and simplifying the critical path, we achieved a stable operating frequency of **50 MHz**.
- **Throughput:** With the 1-stage iterative design taking approximately 11 cycles per block (10 rounds + 1 load/store), the effective throughput is calculated as:

$$\text{Throughput} = \frac{128 \text{ bits}}{11 \text{ cycles}} \times 50 \text{ MHz} \approx 581 \text{ Mbps}$$

While lower than the theoretical maximum of the 10-stage pipeline, this throughput is more than sufficient to saturate the UART interface and meets the requirements for many high-speed embedded applications.

10. Conclusion

10.1 SUMMARY OF ACHIEVEMENTS

This project successfully delivered a fully functional, hardware-accelerated AES-128 encryption system. Our team demonstrated agility in engineering by identifying critical resource bottlenecks in the initial fully pipelined design and successfully pivoting to a robust iterative architecture. The final design integrates a custom communication protocol, a responsive control plane, and a validated cryptographic core into a cohesive system.

10.2 FUTURE WORK

Moving forward, several enhancements could further optimize the system:

Pipelined Unrolling: Exploring a "2-Round" or "5-Round" unrolled loop could offer a middle ground, doubling throughput without the full area penalty of the 10-stage design.

DMA Integration: Replacing the simple UART interface with a Direct Memory Access (DMA) controller over PCIe or AXI could unlock the full potential of the accelerator, removing the communication bottleneck.

Side-Channel Defense: Implementing masking or randomization techniques to protect against power analysis attacks would make the accelerator suitable for high-security physical environments.